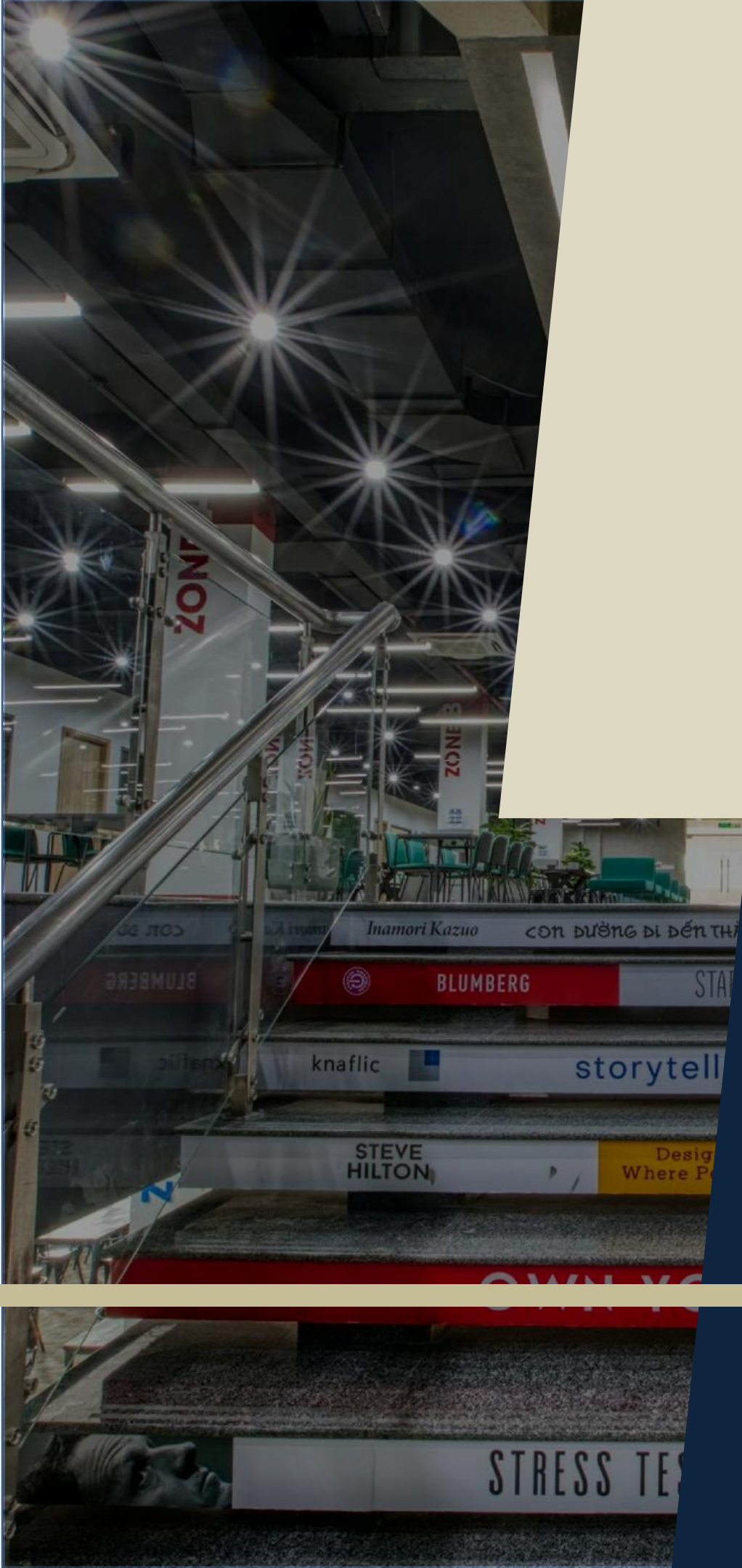


Coding Practice

Architecture



1. Don't Repeat Yourself (DRY)
2. Packages
3. Layers and Tiers
- 4 Architectural styles and patterns

Definition:

- The DRY principle emphasizes the importance of reducing redundancy in code. It suggests that every piece of knowledge or logic should be represented in a single place within a system.

Key Points:

- **Avoid Code Duplication:** Repeating the same logic or code snippet in multiple locations can lead to inconsistencies and errors.
- **Improve Maintainability:** By centralizing code logic, changes can be made in one place, reducing the risk of bugs and making updates easier.
- **Enhance Readability:** A cleaner, more modular design leads to code that is easier to read and understand.

Example:

- Instead of duplicating the same validation logic in different parts of the application, create a reusable function to handle it in one place.

Definition:

- In software architecture, **packages** refer to logical groupings of related classes, components, or modules within a system, organized to manage dependencies, promote reusability, and improve maintainability.

Key Concepts:

- **Logical Grouping:** Packages group related functionality or features into modules to make the system more organized and easier to manage.
- **Separation of Concerns:** Helps to separate different aspects of the system, ensuring that each package focuses on a specific responsibility.
- **Dependency Management:** Packages manage the relationships and dependencies between system components, preventing unnecessary coupling and making dependencies clearer.

Role in Software Architecture:

- **Modularity:** Packages divide the system into smaller, reusable, and independent units that can be easily replaced or updated without affecting the entire system.
- **Encapsulation:** Each package encapsulates its own functionality and hides the internal details from other parts of the system.
- **Scaling:** As systems grow, packages help scale both the development process and the application itself by keeping the architecture organized.

Example: In a Layered Architecture:

- **Presentation Layer Package:** Contains UI components and views.
- **Business Logic Layer Package:** Contains services and business rules.
- **Data Access Layer Package:** Manages data persistence and communication with the database.

Benefits of Using Packages in Software Architecture:

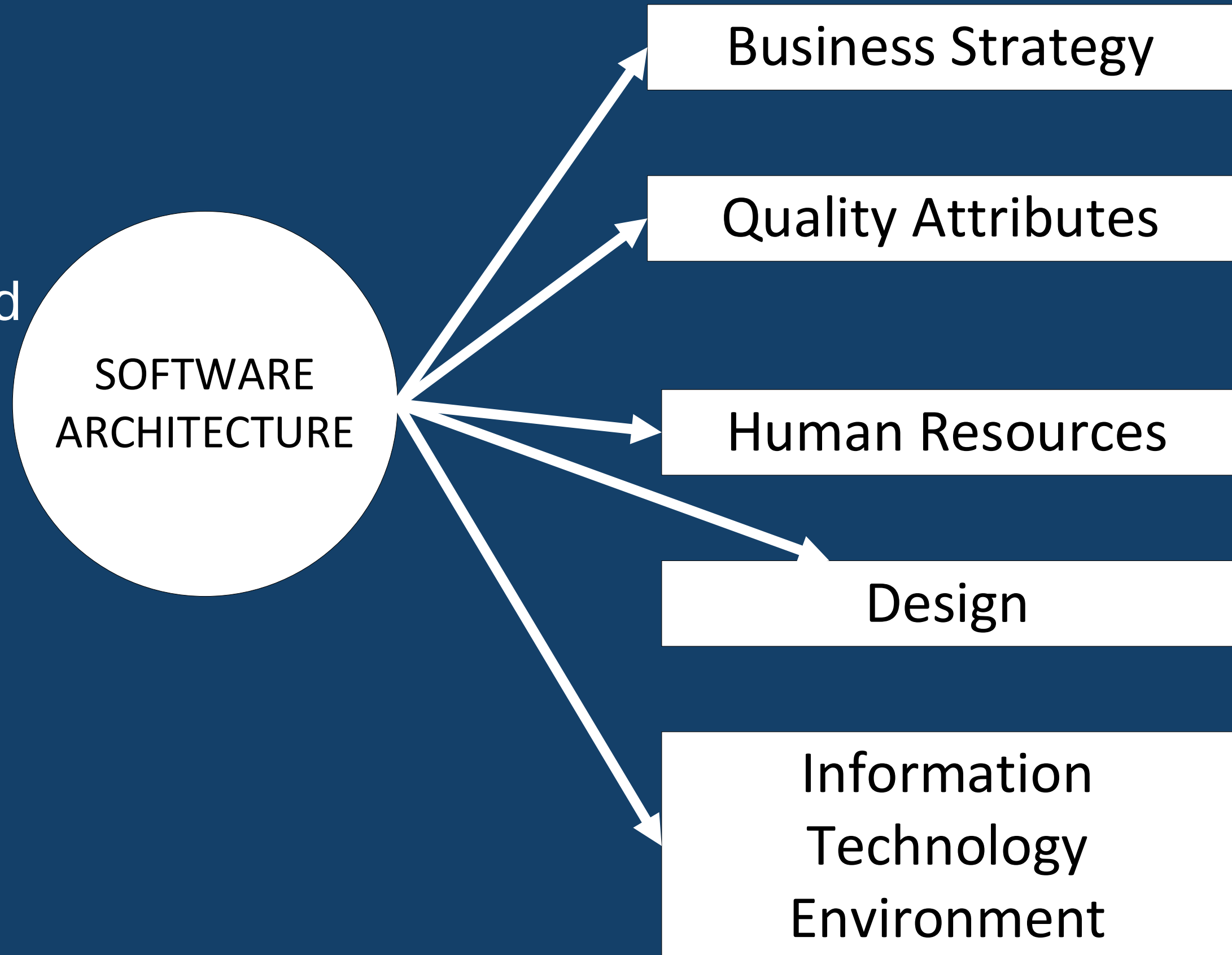
- **Code Reusability:** Enables reuse of packages across multiple projects or parts of a system.
- **Maintainability:** Makes it easier to update or replace parts of the system without disrupting other areas.
- **Clear Structure:** Provides a clear and manageable structure, especially in large systems.

What is Software Architecture?

The book *Software Architecture in Practice* (2012) defines software architecture as "the set of structures needed to reason about the system, which comprises the software elements, the relations among them, and the properties of both."

In other words, software architecture provides a solid foundation for Software Engineers to build software. Each software project begins with a summary, and software architecture is the first step in realizing that summary. Software architecture and design encompass several elements, such as business strategy, quality attributes, human resources, design, and IT environment.

The primary objective of architecture is to define the requirements that influence the structure of the application. A well-structured architecture helps mitigate business risks associated with developing a technical solution and serves as a bridge between business requirements and technical implementation.

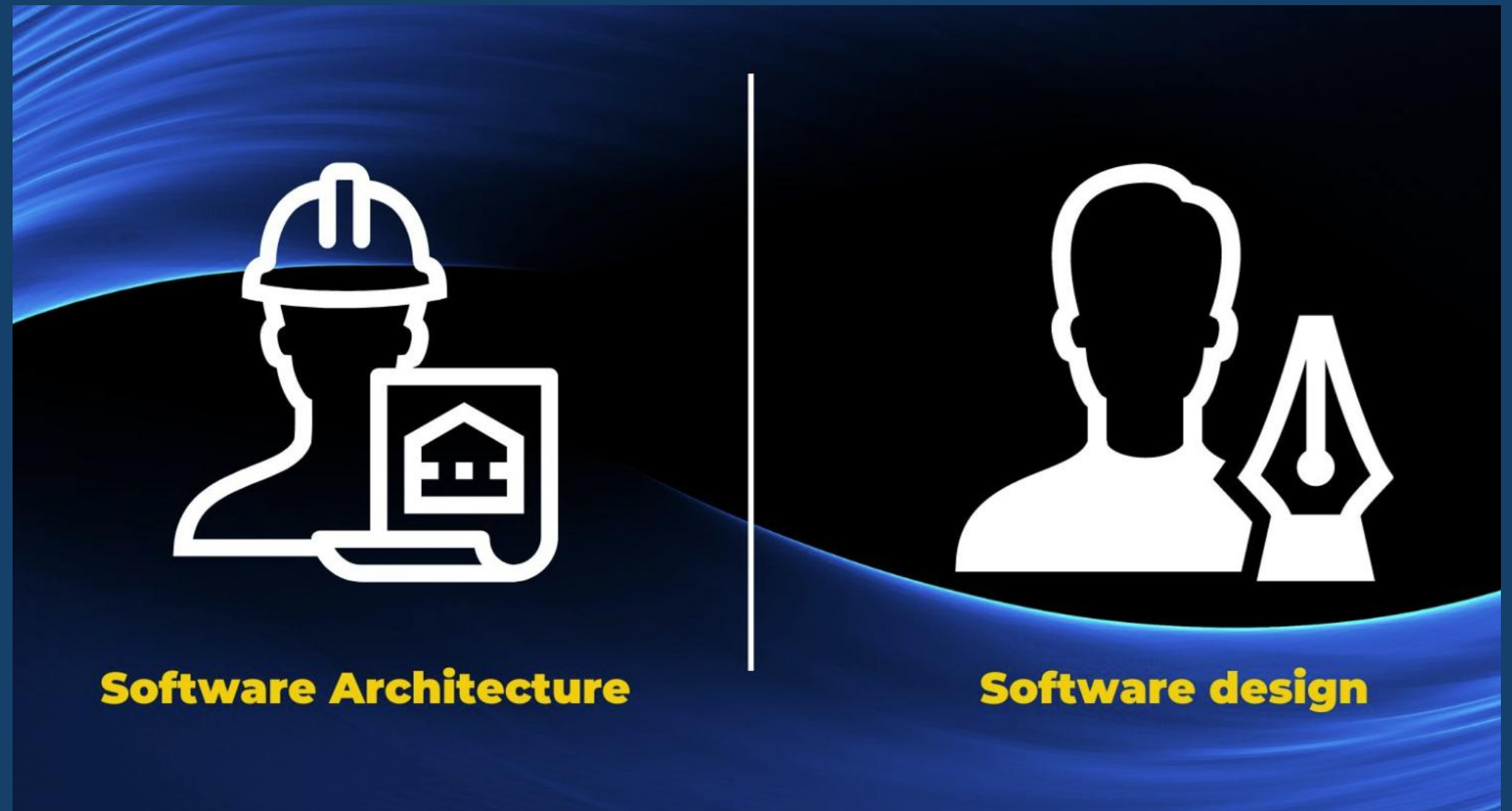


The relationship between software architecture and software design.

Software architecture represents the structure of a system while hiding implementation details. It also focuses on how the elements and components within the system interact with each other. Software design delves deeper into the implementation details of the system. Design concerns include selecting data structures and algorithms or the implementation specifics of individual components.

Architectural and design concerns often overlap. Instead of applying rigid rules to distinguish between architecture and design, you should integrate them. In some cases, certain decisions are clearly more architectural in nature. In other cases, decisions focus more on design and how it helps realize the architecture.

Current trends in software architecture suggest that design evolves over time, and a software architect cannot know everything in advance to fully architect a system. Design often develops during the implementation stages of the system. A software architect must continuously learn and test the design based on real-world requirements.



The role of software architecture

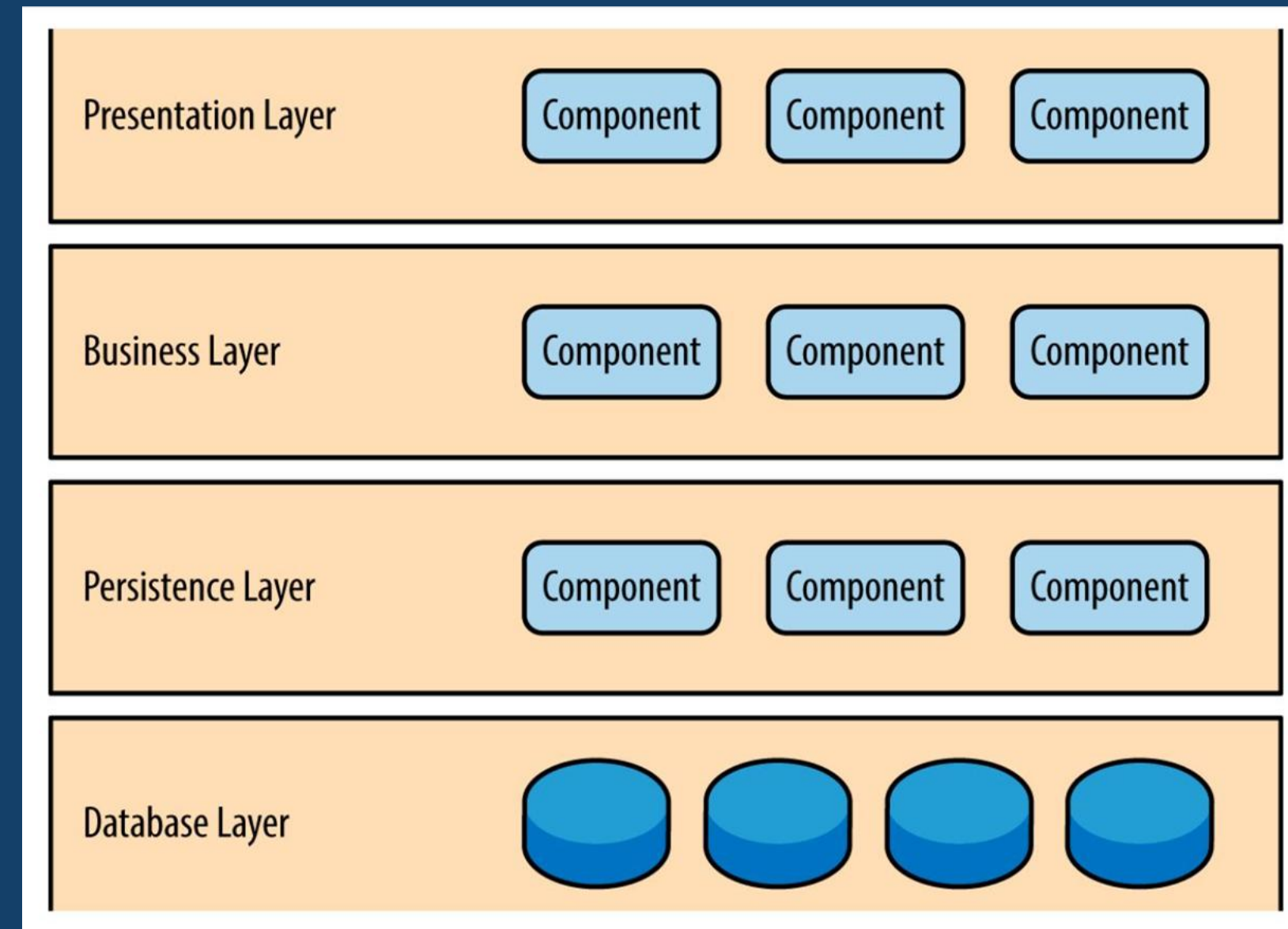
Some key roles of software architecture include:

- Helps stakeholders visualize, understand, and analyze the system.
- Software architecture helps outline the structure of the software.
- It can be used to estimate costs in a project before implementation.
- It serves as a basis for making key decisions to ensure the quality of the work.

Layered Pattern

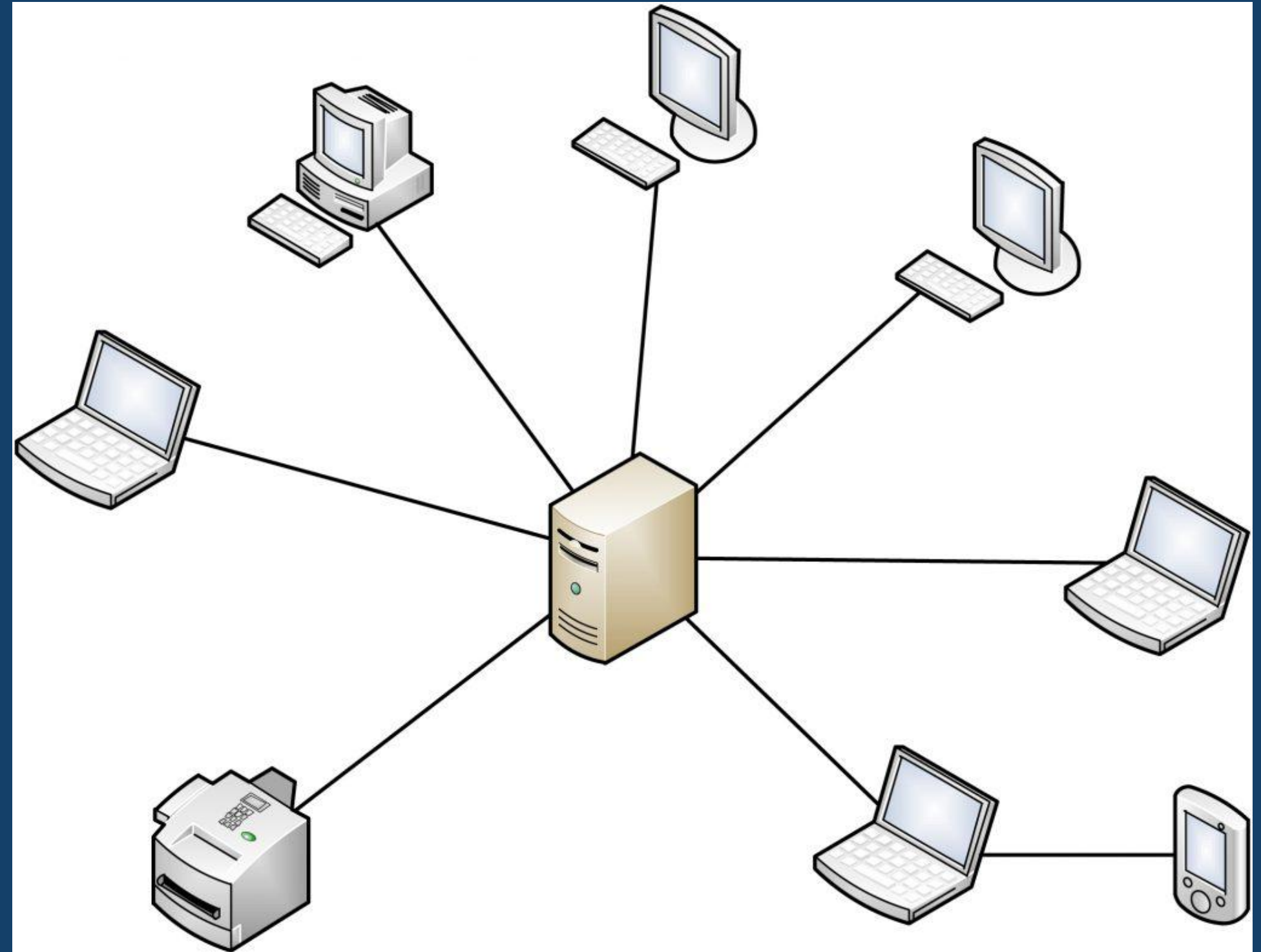
This is the most commonly used pattern for designing most software, notably the e-commerce site Amazon. This pattern is also known as the 'N-tier architecture.' Essentially, it consists of four layers:

- **Presentation Layer** (where users view and input data into the application).
- **Business Logic Layer** (responsible for executing business logic as required).
- **Application Layer** (acts as a communication bridge between the 'presentation layer' and the 'data layer').
- **Data Layer** (contains the database to manage data).



Client–Server Pattern

- This model consists of two main entities: a server and multiple clients. The server holds resources (data, files, or services), and the clients request specific resources from the server. The server then processes the request and responds accordingly.
- This model is suitable for developing software types such as email systems, file-sharing applications, banking systems, etc.

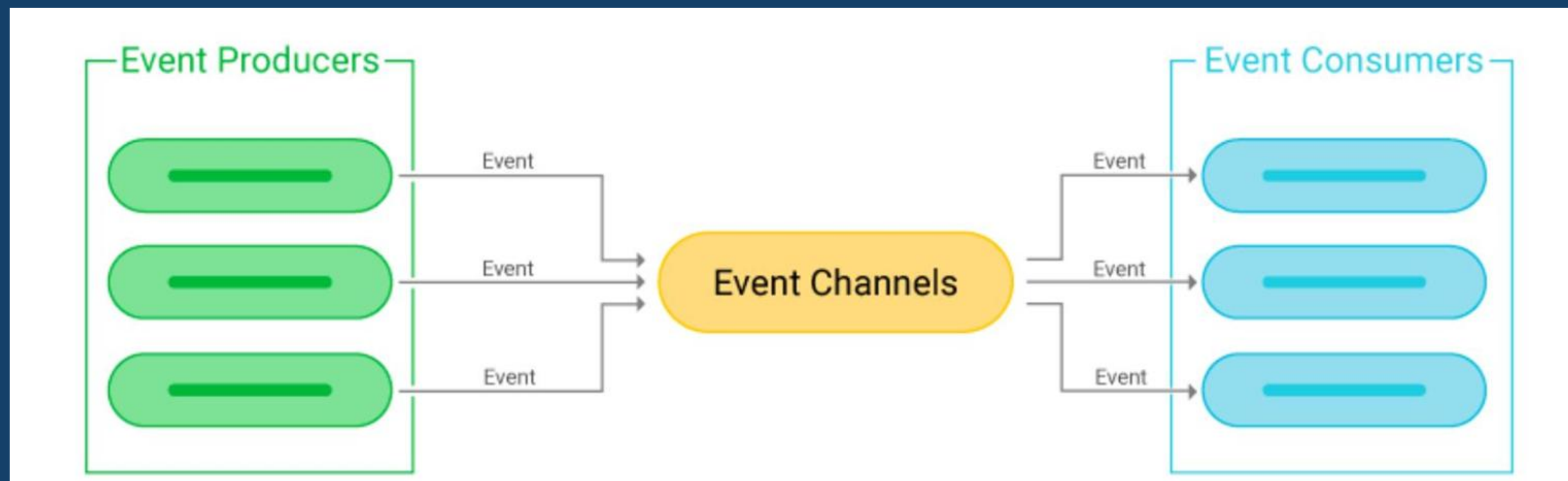


Event-Driven Pattern

Event-driven architecture (EDA) is a software architecture model in which the components or services of a system primarily communicate with each other through the production and consumption of events.

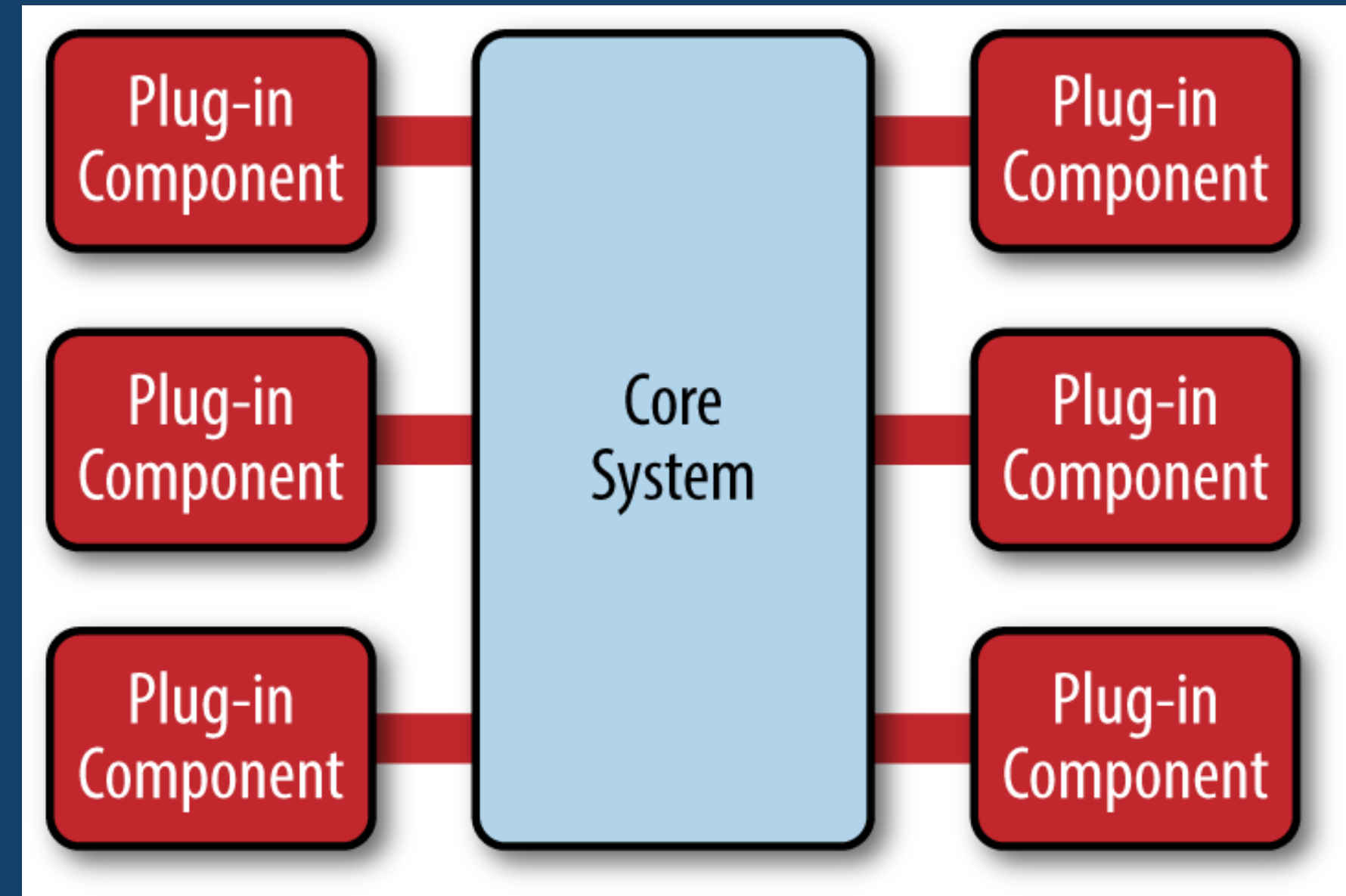
These 'events' can be user actions, data updates, or notifications from other systems. EDA facilitates the creation of flexible, scalable, and easily integrable systems.

EDA is classified based on the topology structure between components in the system, including two common models: Broker topology and Mediator topology.



Microkernel Pattern

- The Microkernel model has two main components: a core system and plug-in modules.
- The core system handles the basic operations of the application. The plug-in modules handle extended functions (such as additional features) and customizations. The microkernel model is primarily favored for developing product-based applications and scheduling applications. For example, Instagram, YouTube Shorts...



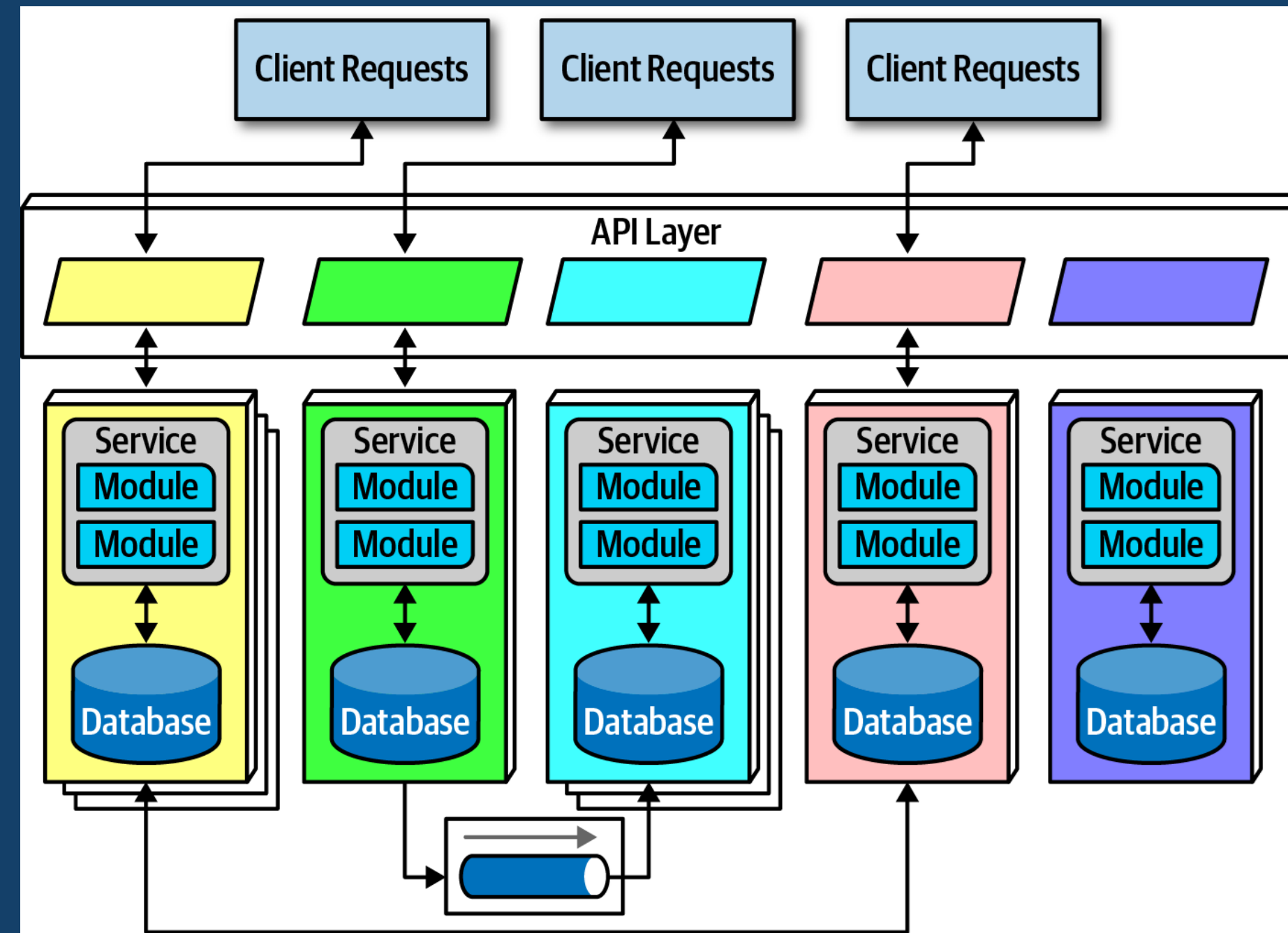
Source: Fundamentals of Software Architecture - Mark Richards

Microservices Pattern

The Microservices model is a collection of small services combined to create a real application. Instead of building a large application, small programs are built for each service (function) of the application independently. These small programs are then aggregated to form a complete application.

The modules in Microservices applications are loosely coupled. Therefore, you can easily modify and extend them.

Netflix is one of the most common examples of a Microservices architecture with integrated software.



Source: Fundamentals of Software Architecture - Mark Richards

Start your future at EIU

Q&A

Start your future at EIU

THANK YOU